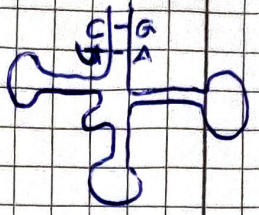
GIK

T->V

RNA secondary structure prediction



→ RNA folds

→ RNA Sequence:

a string over the alphabet $A = \{A, C, G, U\}$ // 100 - 1000

Secondary Structure:

RNA sequence of length n, $w \in A^n$. A secondary structure of w is a set $S_w \subseteq \{1,..,n\} \times \{1,..,n\}$ satisfying conditions:

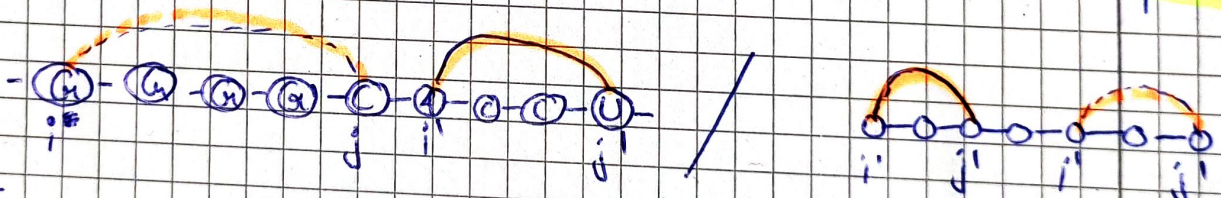
1) $1 \leq i < j \leq n$, $\forall (i,j) \in S_w$ // aufsteigend sortiert

2) For different $(i,j), (i',j') \in S_w$, we either have $j < i'$ or $j' < i$ or $i' < i < j < j'$ or $i < i' < j < j'$

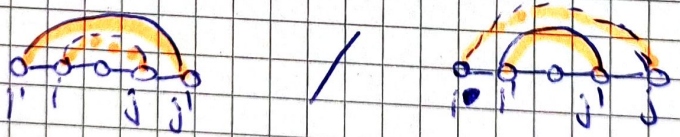
Each position at most in one pair

3) $\forall (i,j) \in S_w$, $(w[i], w[j]) \in \{(A,U), (U,A), (G,C), (C,G), (G,U), (U,G)\}$

4) $\forall (i,j) \in S_w$, we have $j - i - 1 \geq l_{min}$, where l_{min} is a user defined constant, the minimal loop size



or



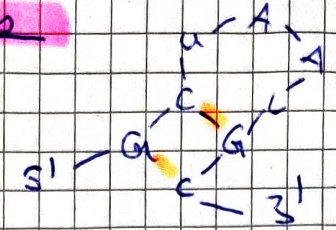
↳ no knots! ($i < i' < j < j'$) ⇒ RNA secondary structure as tree/graph

- $(i,j) \in S_w$ is called base pair.
- RNA twisted to form base-pairs
- l_{min}, minimal number of bases between the base pairing positions
  ↳ eg: $l_{min} = 4$

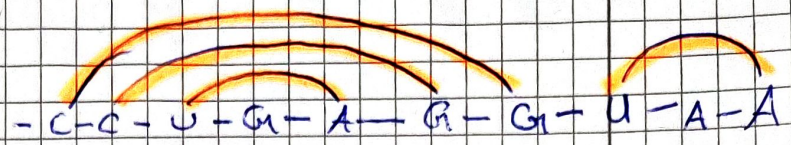
GUK

Representation of secondary structures

1) Base pair graph:



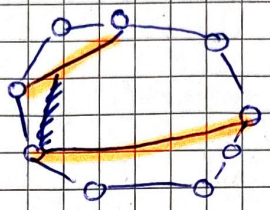
2) Mountain plot:



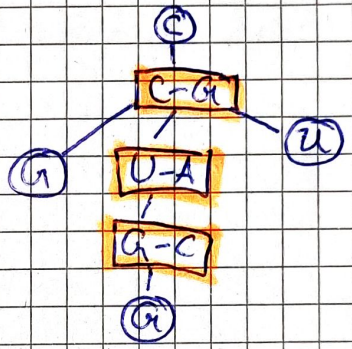
3) Vienna Notation:

```
C C C G U G A A A U U G G A
. ( ( . . ( . . . . ) ) )
```

4) Circle Plot:



5) Tree representation:



(23) Vienna Notation

. ((. ((. . .)) . ((. . .)) .))

↳ [(2,22), (3,21), (5,11), (6,10), (13,19), (14,18)]

```
viennapairlist(viennastring):
    stack = []
    pairlist = []
    for idx, c in enumerate(viennastring):
        if c == '(':
            stack.append(idx+1)
        elif c == ')':
            pairlist.append((stack.pop(), idx+1))
    pairlist.sort()
    return pairlist
```

GIK

Minimize the free Energy

→ Goal: Determine optimal secondary structure

↳ Model: A molecule has a free energy, which determines the potential of the molecule to form bindings, thus its energy

→ Find secondary structure with minimal free energy

→ Function to attribute energy values to a structure

Independent base pair

→ Nussinov - Algorithm

→ Assumes: In any RNA structure the energy of a base-pair is independent of all the others

Total free energy:

$$Efe(S_w) = \sum_{(i,j) \in S_w} \alpha(w[i], w[j])$$

$\alpha(x, y)$ is the free energy of pairing base x and y

→ base pairs are stabilizing a structure → $\alpha(x,y) < 0$

→ compute $n \times n$-table E

E(i,j): minimum free energy of all secondary structures for $w[i..j]$, $1 \leq i < j \leq n$

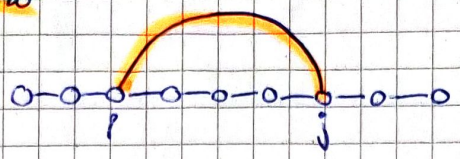
→ E(1,n): minimum free energy of all secondary structures for w

GdK

Independent Base Pairs

S_w for $w[i..j]$ with min free energy. Two Cases:

Case 1: $(i,j) \in S_w$



→ we obtain the energy contribution $\alpha(w[i], w[j])$

→ No base pair for $i+1$ to $j-1$ with any base k, $k < i$ or $k > j$ // knots ↯

↳ remaining base pairs in range $[i+1, j-1]$

↳ remaining part of the optimal structure for $w[i..j]$ must be determined for $w[i+1..j-1]$

$$E(i,j) = E(i+1, j-1) + \alpha(w[i], w[j])$$

Case 2: (i,j) do not form a base pair in S_w

2a: j is not involved in a base-pair.

minimal energy value is determined from $w[i..j-1]$,

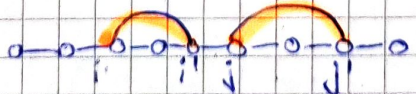
$$E(i,j) = E(i,j-1)$$

2b: i is not involved in a base pair.

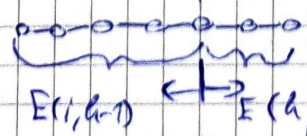
consider $w[i+1..j]$

$$E(i,j) = E(i+1, j)$$

2c: both i and j form base-pairs (but not with each other), $(i,i') \in S_w$, $(j,j') \in S_w$



$$E(i,j) = \min\{ E(i, k-1) + E(k,j) \mid i+2 \le k \le j-1 \}$$



Independent base pair

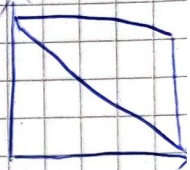
All together

$$E(i,j) = \min \begin{cases} E(i+1,j-1) + \alpha(w[i], w[j]), \\ E(i+1,j), \\ E(i,j-1), \\ \min\{ E(i,k-1) + E(k,j) \mid i+2 \le k \le j-1 \} \end{cases}$$

→ $E(i,j)$ depends on values in previous diagonals

Dependencies:

E-Term	Constraint	on diagonal
$E(i+1,j-1)$		$j-i-2$
$E(i+1,j)$		$j-i-1$
$E(i,j-1)$		$j-i-1$
$E(i,k-1)$	$k \le j-1$	$k-i-1 \le j-i-2$
$E(k,j)$	$i+2 \le k$	$j-k \le j-i-2$



$E(i,j)$ depends on all $j'-i' < j-i$, $E(i',j')$

↳ Diagonal-wise Evaluation

In: Sequence $w \in A^n$ and energy function $\alpha: A \times A \to \mathbb{R}$
Out: Min free energy & secondary structures of w

```
for i ← 1 upto n do
    E(i,i) ← 0          // on diagonal 0
end for
for l ← 2 upto n do                 // l − 1 = j − i
    for i ← 1 upto n−l+1 do         // recalculate diagonal l−1
        j ← i + l − 1               // substring w[i..j] of length l
        if j − i − 1 < l_min then   // loop size too small?
            E(i,j) ← 0
        else
            compute E(i,j) according that equation above
        end if
    end for
end for
return E(1,n)
```

Space: $O(n^2)$, Runtime: $O(n^3)$

Independent Base Pair

In: w, α, E

Out: secondary structure with minimum free energy

Initial call: traceback(1, n)

```
traceback (i,j):
  if i < j then
    if E(i,j) = E(i+1, j-1) + α(w[i], w[j]) then
      print (i,j)
      traceback (i+1, j-1)
    else
      if E[i,j] = E(i+1,j) then
        traceback (i+1, j)
      else if E(i,j) = E(i, j-1) then
          traceback (i, j-1)
      else
        for k ← i+2 upto j-1 do
          if E(i,j) = E(i, k-1) + E(k, j) then
            traceback (i, k-1)
            traceback (k, j)
            break
          endif
        endfor
      endif
    endif
  endif
```

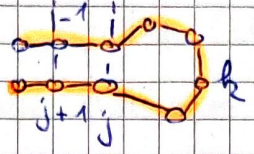
Structures with loops

→ Idea: free energy depends on adjacent base-pairs

Def:

RNA-sequence w, secondary structure S_w, $(i,j) \in S_w$

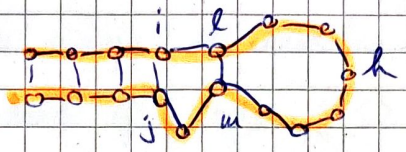
→ a position k, $k \in [1,n]$ is accessible from (i,j),
if $i < k < j$ and no $(i',j') \in S_w$ exists s.t. $i < i' < k < j' < j$



→ k is accessible from (i,j) and from $(i-1,j+1)$

→ (i,j) is accessible from $(i-1,j+1)$

→ a base-pair (l,m) is accessible from (i,j), if $i < l < m < j$
and no $(i',j') \in S_w$ exists s.t. $i < i' < l < m < j' < j$

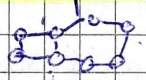


→ (l,m) is accessible from (i,j)

Loops

For a $(i,j) \in S_w$, a loop is the set of all bases accessible from (i,j):

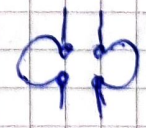
→ $(i,j) \in S_w$ closes a hairpin loop if no base-pair is accessible from (i,j)



→ $(i,j) \in S_w$ closes a bulge loop on i, if there is a base-pair $(i', j-1) \in S_w$ for some $i' > i+1$,

→ $(i,j) \in S_w$ closes a bulge loop on j, if there is a base-pair $(i+1, j') \in S_w$ for some $j' < j-1$

→ $(i,j) \in S_w$ closes an interior loop if there is a base-pair $(i',j') \in S_w$ s.t. $i+1 < i' < j' < j-1$



size of a loop closed by (i,j) = # of unpaired bases accessible from (i,j)

RNA

Structures with loops

→ base-pair → loops of size 0

→ helical region, rows of base pairs

↳ stem $(i,j),(i+1,j-1)\ldots$ ⇒ stacked pairs

→ # loops in a secondary structure = # of base-pairs if contains

→ helical region has negative (stabilizing) free energy, while loops of size greater than zero have positive (destabilizing) free energy

→ total free energy = $\sum_{\text{loops}}$ free energies of loop

→ $V_{i,j}$ = min free energy structure for $w[i+1,\ldots,j-1]$, is a subset of $\{i+1,\ldots,j-1\} \times \{i+1,\ldots,j-1\}$

Functions:

→ $\alpha(x,y)$, free energy of a base pair (x,y)

→ $\zeta(k)$, destabilizing free energy of a hairpin loop of size k

→ η, stabilizing free energy of adjacent base pairs

→ $\beta(k)$, destabilizing free energy of a bulge of size k

→ $\gamma(k)$, destabilizing free energy of an interior loop of size k

efe($V_{i,j}$):

1) $V_{i,j}$ is hairpin loop ⇒ $efe_h(i,j) = \alpha(w[i],w[j]) + \zeta(j-i-1)$

2) $V_{i,j}$ is helical region ⇒ $efe_r(i,j) = \alpha(w[i],w[j]) + \eta + E(i+1,j-1)$

3) $V_{i,j}$ is bulge loop on i ⇒ $efe_{bli}(i,j) = \alpha(w[i],w[j]) + \min_{k \in [1,j-i-2]} \{\beta(k) + E(i+k+1,j-1)\}$

4) $V_{i,j}$ is bulge loop on j ⇒ $efe_{blj}(i,j) = \alpha(w[i],w[j]) + \min_{k \in [1,j-i-2]} \{\beta(k) + E(i+1,j-k-1)\}$

5) $V_{i,j}$ is interior loop ⇒

$$efe_{il}(i,j) = \alpha(w[i],w[j]) + \min\{\gamma(k_1+k_2) + E(i+1+k_1,j-1-k_2)\}$$

$$k_1 \geq 1,\ k_2 \geq 1,\ k_1+k_2 \leq j-i-2\}$$

GTK

Structures with Loops

→

$$E(i,j) = \min \begin{cases} E(i+1,j), & \text{if } i \text{ is not accessible from a base pair} \\ E(i,j-1), & \text{if } j \text{ is not accessible from a base pair} \\ \min\{E(i,k-1) + E(k,j) \mid i+2 \le k \le j-1\} \\ \min\{e_{stk}(i,j), e_{hl}(i,j), e_{bl}(i,j), e_{bl}(i,j), \\ \quad e_{il}(i,j)\} \end{cases}$$

→ for computation just change the recurrences

→ Runtime: helical region / hairpin $O(n^2)$
bulge loop $O(n^3)$
interior loop $O(n^4)$
} total runtime $O(n^4)$

→ maximal loop size reduces runtime to $O(n^3)$, case three needs $O(n^3)$
↳ since bulge / interior only constant comparisons

Improvements

→ Both Algs only predict one structure ☹

GIK – Ü3

3.1) Anzahl der potentiell möglichen RNA-Sekundär-Strukturen einer Sequenz der Länge $n \geq 1$.
↳ $S(n)$

```
rna_count_rec(n)
    if n==1 or n==2 : return 1
    prod_sum = 0
    for i in range(1, n-2):      // i ∈ [1, n-3]
        prod_sum = prod_sum + rna_count_rec(i) * rna_count_rec(n-2-i)
    return rna_count_rec(n-1) + rna_count_rec(n-2) + prod_sum
```

n :	# Strukturen
1 ○	→ 1
2 ○–○	→ 1
3 ○–○–○	→ 2
4	→ 4
5	→ 8

Cond for Sekundärstruktur:

I) $1 \leq i < j \leq n$

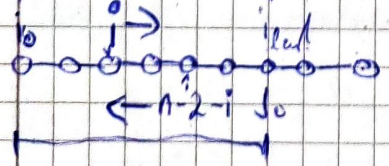
II) $(i,j), (i',j') \in S_{RNA}$
$j < i'$ or $j' < i$ or $i' < i < j'$ or $i < i' < j' < j$

III) irelevant here

IV) $j - i - 1 \geq l_{min}$

→ $n = 1$ or $n = 2$ return 1, since $l_{min} = 1$ only nested chains possible

→ $i \in [1, n-3]$: // since $l_{min} = 1$ we need 3-nucleotides for a new stable Pair $(i, n-2-i)$



// split structures into two calculable # of Structures
→ then $|A \times B|$ all combinations multiply both
→ "*2" case ○–○○○ or ○–○–○○ possible

→ return rna_count_rec(n-1) + rna_count(n-2) + prod_sum

wurden ~~Both are not in loop~~ klar

GAK

Fast computation of the unit Edit Distance

$$edist_\delta(u,v) = \min\{\delta(A) \mid A \text{ is an alignment of } u \text{ and } v\}$$

$$edist_\delta(u,v) = E_\delta(m,n)$$

$$E_\delta(i,j) = \begin{cases} 0 & , i=0 \ \& \ j=0 \\ E_\delta(i-1,0) + \delta(u[i] \to \varepsilon) & , i>0 \ \& \ j=0 \\ E_\delta(0,j-1) + \delta(\varepsilon \to v[j]) & , i=0 \ \& \ j>0 \\ \min \begin{cases} E_\delta(i-1,j) + \delta(u[i] \to \varepsilon), \\ E_\delta(i,j-1) + \delta(\varepsilon \to v[j]), \\ E_\delta(i-1,j-1) + \delta(u[i] \to v[j]) \end{cases} & , i>0 \ \& \ j>0 \end{cases}$$

$$\delta(a \to b) = \begin{cases} 0 & , a=b \\ 1 & , \text{else} \end{cases}$$

Recurrence for unit edit distance

$$E_\delta(i,j) = \begin{cases} 0 & , i=0 \ \& \ j=0 \\ i & , i>0 \ \& \ j=0 \\ j & , i=0 \ \& \ j>0 \\ \min \begin{cases} E_\delta(i-1,j)+1, \\ E_\delta(i,j-1)+1, \\ E_\delta(i-1,j-1) + \\ \quad (\text{if } u[i]=v[j] \text{ then } 0 \text{ else } 1) \end{cases} & , i>0 \ \& \ j>0 \end{cases}$$

Consecutive entries in E_δ-columns, E_δ-rows and E_δ-diagonals differ by at most one. The entries in E_δ-diagonals are non-decreasing.

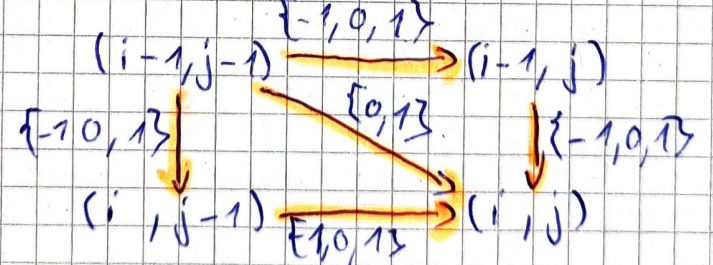


Table E_δ has properties:

$$-1 \le E_\delta(i,j) - E_\delta(i-1,j) \le 1, \quad \forall i,j, \ i \in [1,m], j \in [0,n]$$

$$-1 \le E_\delta(i,j) - E_\delta(i,j-1) \le 1, \quad \forall i,j, \ i \in [0,m], j \in [1,n]$$

$$0 \le E_\delta(i,j) - E_\delta(i-1,j-1) \le 1, \quad \forall i,j, \ i \in [1,m], j \in [1,n]$$

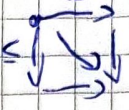
Properties of the unit edit distance

Lemma 8:

$\forall\, i \in [1,m], \forall\, j \in [1,n]$, the following holds:

If $E_g(i-1,j-1) \leq E_g(i,j-1)$ and $E_g(i-1,j-1) \leq E_g(i-1,j)$, then

$$E_g(i,j) = E_g(i-1,j-1) \iff x[i] = y[j]$$



Proof: recurrence will be reduced to $\leq$ since $(\leq +1) \Rightarrow <$

$$E_g(i,j) = \min \left\{ \begin{array}{l} E_g(i,j-1)+1, \\ E_g(i-1,j)+1, \\ E_g(i-1,j-1) + (\text{if } x[i]=y[j] \text{ then } 0 \text{ else } 1) \end{array} \right\}$$

$$= E_g(i-1,j-1) + (\text{if } x[i]=y[j] \text{ then } 0 \text{ else } 1)$$

Hence $E_g(i,j) = E_g(i-1,j-1) \iff (\text{if } x[i]=y[j] \text{ then } 0 \text{ else } 1) = 0$

$$\iff x[i] = y[j]$$

∎

Lemma 9:

$\forall\, i \in [1,m], \forall\, j \in [1,n]$, the simplified recurrence holds:

$$E_g(i,j) = \begin{cases} E_g(i-1,j-1) & \text{if } x[i] = y[j] \\ 1 + E_g(i-1,j) & \text{, else if } E_g(i-1,j) \leq E_g(i-1,j-1) \\ 1 + \min\{E_g(i,j-1), E_g(i-1,j-1)\} & \text{, else} \end{cases}$$

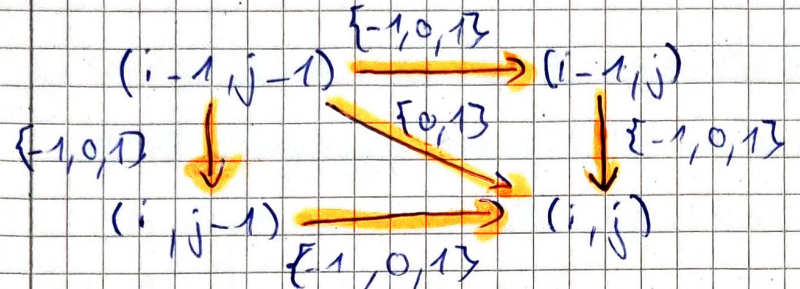
GIK - Ü3

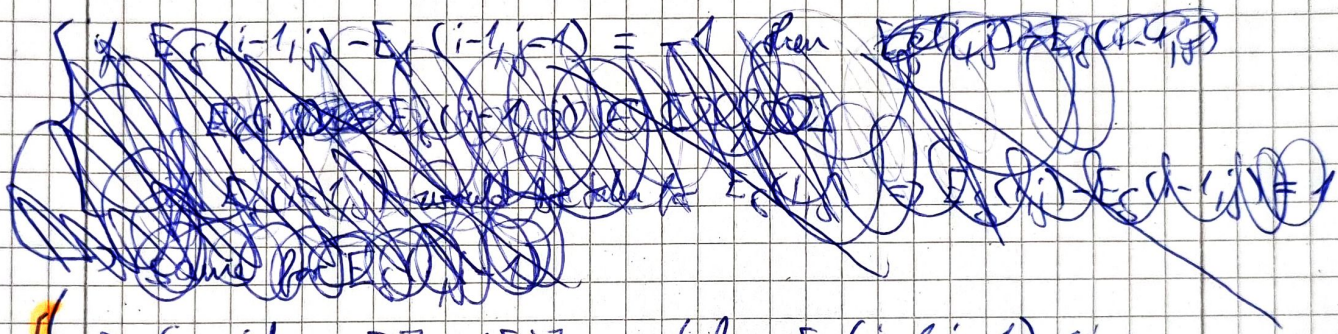
3.1) $i,j > 0$

$$E_\delta(i,j) = \begin{cases} E_\delta(i-1,j-1), & \text{if } u[i] = v[j] \\ 1 + E_\delta(i-1,j), & \text{elif } E_\delta(i-1,j) < E_\delta(i-1,j-1) \\ 1 + \min\{E_\delta(i,j-1), E_\delta(i-1,j-1)\} & \text{else} \end{cases}$$

Nachbar Einträge E_δ:

$$\begin{aligned} -1 \leq E_\delta(i,j) - E_\delta(i-1,j) &\leq 1, \quad i \in [1,m], j \in [0,n] \\ -1 \leq E_\delta(i,j) - E_\delta(i,j-1) &\leq 1, \quad i \in [0,m], j \in [1,n] \\ 0 \leq E_\delta(i,j) - E_\delta(i-1,j-1) &\leq 1, \quad i \in [1,m], j \in [1,n] \end{aligned}$$





↳ So if $u[i] = v[j]$ one takes $E_\delta(i-1,j-1)$ since then $E_\delta(i,j) - E_\delta(i-1,j-1) = 0$

Case 1 ↳ if $E_\delta(i-1,j) / E_\delta(i,j-1) - E_\delta(i-1,j-1) = -1$ and one would take $E_\delta(i-1,j) / E_\delta(i,j-1)$ as minimum $\Rightarrow E_\delta(i,j) - E_\delta(i-1,j) / E_\delta(i,j-1) = 1$ because of indel cost = 1.

↳ then $E_\delta(i,j) - E_\delta(i-1,j-1)$ would still be $= 0$. ▨

Case 2 ⇒ if $u[i] \neq v[j]$ and $E_\delta(i-1,j) < E_\delta(i-1,j-1) \rightarrow E_\delta(i-1,j) - E_\delta(i-1,j-1) = -1$ one do not need to consider $E_\delta(i-1,j-1)$ since will edit distance.

It doesn't matter if we take $E_\delta(i-1,j)$ or $E_\delta(i,j-1)$ (if $E_\delta(i,j-1) < E_\delta(i-1,j-1)$ also) since if $E_\delta(i,j-1) < E_\delta(i-1,j-1) \Rightarrow E_\delta(i,j-1) - E_\delta(i-1,j-1) = -1$ so same difference + 1 same result! ▨

Case 3 is self explaining ▨

A Lazy evaluation Strategy

→ no need to evaluate E_δ completely

→ Only calculate: → Diagonal, if $x[i] = y[j]$

→ $E_\delta(i-1, j)$

→ if $E_\delta(i-1, j) \leq E_\delta(i-1, j-1)$ no need to compute $E_\delta(i, j-1)$ else

→ $E_\delta(i, j-1)$

→ A request to evaluate $E_\delta(m, n)$ then triggers the computation of all necessary values in E_δ in a band around the main diagonal, the smaller $E_\delta(m, n)$ the smaller the band

→ Solved using a stack

GAK – Ü3

(3.3) Lazi Editdistance

$$E_S(i,j) = \begin{cases} E_S(i-1,j-1) & , u_{i-1} = v_{j-1} \\ 1 + E_S(i-1,j) & , E_S(i-1,j) < D \\ 1 + \min\{E_S(i,j-1), E_S(i-1,j-1)\} & , \text{else} \end{cases}$$

```
lazy_dp(u, v):
    mdict = { }
    stack = [ ]
    m     = len(u)
    n     = len(v)

    stack.append((m, n))          //
    while stack:
        u_i, v_i = stack.pop()

        if u_i == 0:              // case 1: i=0 → j
            mdict[(u_i, v_i)] = v_i

        elif v_i == 0:            // case 2: j=0 → i
            mdict[(u_i, v_i)] = u_i

        elif (u_i-1, v_i-1) not in mdict:   // compute diag entry first
            stack.append((u_i, v_i))
            stack.append((u_i-1, v_i-1))

        elif u[u_i-1] == v[v_i-1]:          // case 3: chars are the same
            mdict[(u_i, v_i)] = mdict[(u_i-1, v_i-1)]

        elif (u_i-1, v_i) not in mdict:     // first compute left entry
            stack.append((u_i, v_i))
            stack.append((u_i-1, v_i))

        elif mdict[(u_i-1, v_i)] < mdict[(u_i, v_i-1)]:   // case 4: Δ < D
            mdict[(u_i, v_i)] = mdict[(u_i-1, v_i)] + 1

        elif (u_i, v_i-1) not in mdict:     // first compute upper
            stack.append((u_i, v_i))
            stack.append((u_i, v_i-1))

        elif mdict[(u_i, v_i-1)] < mdict[(u_i-1, v_i-1)]:
            mdict[(u_i, v_i)] = mdict[(u_i, v_i-1)] + 1     // case 5.1: U < D

        else:
            mdict[(u_i, v_i)] = mdict[(u_i-1, v_i-1)] + 1   // case 5.2: D < U

    return mdict
```

GLK

Diagonalwise Computation

→ Consider only which band around the main diagonal of E_δ is actually evaluated

→ Output-Sensitive algorithm = Running time depends on the distance value

↳ smaller distance → faster

d-path:

→ $d \in \mathbb{N}_0$

→ A d-path is a path in $G(V,V)$ which contains exactly d edges with cost 1

→ d-path has cost d

→ A d-path that begins at $(0,0)$ is an initial d-path

→ A d-path that doesn't begin at $(0,0)$ is an internal d-path

↳ we assume initial d-paths

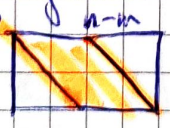
forward diagonal:

→ $k \in [-m, n]$

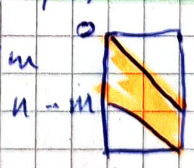
→ forward diagonal k ($\searrow$) consists of all pairs (i,j), $i \in [0,m]$, $j \in [0,n]$, $j - i = k$

$(0,0)$ is on diagonal 0, and (m,n) is on diagonal $n-m$.

$n > m$



$n < m$



Gotoh

Diagonal-wise Computation

Lemma:

A d-path must end on a diagonal $h \in D_d$,

$$D_d = \{-d, -d+1, \ldots, d-1, d\}$$

↳ considers worst case

An output sensitive algorithm

maximal d-path:

A d-path of maximal length ending on diagonal h is called a maximal d-path on h

↳ For $\forall d = 0, 1, \ldots$ and all h, $-d \leq h \leq d$

compute the endpoint of a maximal d-path on h

↳ these are all diagonals we need to consider

→ For a minimal d s.t. (m, n) is the endpoint of a maximal d-path on u-m, we have $\text{edist}_S(u, v) = E_S(m, n) = d$

Def:

$$j - i = h \iff j = h + i$$

$\forall d \in \mathbb{N}_0$ and any $-d \leq h \leq d$

$$\text{front}(h, d) = \max\{\, i \mid 0 \leq i \leq m,\ E_S(i, h+i) = d \,\}$$

h: diagonal; i: row number; $E_S(i, h+i)$: on diagonal h; d: difference

↳ endpoint of a maximal d-path

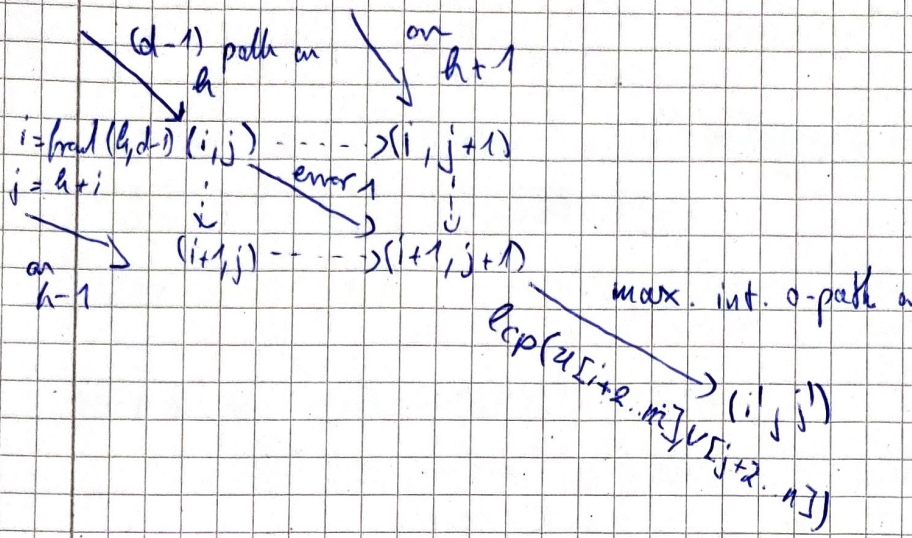
GIK

~~Diff~~

An output sensitive Alg

Lemma:

Let $d_{min} = \min\{ d \in \mathbb{N}_0 \mid -d \le n-m \le d, \text{front}(n-m, d) = m \}$,
then d_{min} is the unit edit distance of u and v



Recurrence front:

$$\text{front}(k,d) = \ell + 1 \mid lcp(u[\ell+1 \ldots m], v[k+\ell+1 \ldots n]) \mid$$

$$\text{where } \ell = \max \begin{cases} \text{front}(k-1, d-1), & \rightarrow \text{// we are coming from left} \\ \text{front}(k+1, d-1)+1 & \downarrow \text{// } \\ \text{front}(k, d-1)+1 & \searrow \text{// } \end{cases} + i + 1$$

Alg: in: sequence u & v
out: $\text{edist}_g(u,v)$

```
function fastedit (u, v)
    (m, n) ← (|u|, |v|)
    for d ← 0 upto max{n,m} do
        for k ← -d upto d do
            Compute front(k,d) with
            either
            if -d ≤ n-m ≤ d and front(n-m, d) = m then
                return d
            end if
        end for
    end for
end function
```

An output sensitive Algorithm

$e = edist_S(u,v)$; For each d, $d \in [0,e]$, the alg. computes $2 \cdot d + 1$ values.

$$\sum_{d=0}^{e} (2d+1) = e+1+2\sum_{d=0}^{e} d = e+1+2\frac{(e+1)\cdot e}{2}$$
$$= e+1+e^2+e \quad \in O(e^2)$$

→ lcp calculation at most $O(m+n)$ pairwise character matches and $O(e^2)$ mismatches.
↳ $O(m+n+e^2)$ → also complete Runtime

→ each generation of front-values
front$(-d,d), \dots,$ front(d,d) , $d>0$
can be computed from the previous generation.
↳ only need to store two generations at any time, requiring $O(e)$ space

→ For getting optimal alignment, mark from where you came and keep track of lcp values
→ each front-entry stores 3 bits and an integer → constant space
→ We need to keep track of all generations $O(e^2)$ space
↳ Runtime $O(m+n+e^2)$

Q1K

Optimizing the output sensitive Alg.

→ previous Alg: computes $2e+1$ front values in last generation, if $edist \leq e$.

→ Old prev: Banded Alignment Methoden → saved edist vals to be computed by
↳ save three optimal Alignment boarder front

⇒ Dynamically restrict the number of front entries in front$(-d,d), \ldots,$ front(d,d) to be computed.

↳ depends on editing row value in the front entries of the current & previous generation

↳ user provides a lower bound on the identity of the Alignment to be computed

Larger minimal identity → allow to more aggressively reduce the number of front-entries to be computed

Two Strategies:

1) based on the regional alignment quality for each front-entry, front(k,d)
↳ determined by the number of matches in the last k_y columns of the Alignment represented by front(k,d)
↳ $k_y \leq 64$, the history size

→ the less matches in the last k_y columns, the poorer the regional alignment quality and the more dense neighboring fronts are

→ each front entry is associated with a bitvector of k_y bits
↳ a window of size k_y

→ each edit-operation leading to a new front-entry triggers a left-shift
↳ each match a 1-bit is added on the right

GapK

Optimizing the output sensitive alg.

Two Strategies:

1) One defines a threshold $r_q \leq k_y$ and trims all front-entries with a regional alignment quality less than r_q

   e.g.: $k_y = 60$, $\text{minid} = 70$, $\Rightarrow r_q = \frac{\text{minid} \cdot k_y}{100} = 42$

⇒ front-entry consists of three values:

- → integer row = front(k, d)
- → bitvector b of k_y bits storing for each of the previous k_y edit-operations in the regional alignment ending at (i,j) with $i = \text{front}(k,d)$ and $j = i+k$ whether it is a match or not
- → # of matches nm in regional alignment

```
evaluate frontentry (u, v, k, d)
    ℓ ← max { FE(k-1, d-1).row
              FE(k+1, d-1).row + 1
              FE(k, d-1).row + 1 }
    if ℓ = FE(k-1, d-1).row:
        fe ← FE(k-1, d-1)
    else if ℓ = FE(k+1, d-1).row + 1:
        fe ← FE(k+1, d-1)
    else
        fe ← FE(k, d-1)
    endif
    if fe.b[0] is set:                  // Match at start of regional alignment
        fe.nm ← fe.nm - 1               // track error where match count
    endif
    fe.b ← fe.b << 1                    // shift window to left
    (i,j) ← (ℓ+1, k+ℓ+1)                // Now track 0-path on k
    while i ≤ m and j ≤ n and u[i] = v[j] do    // determine LCP
        if fe.b[0] is not set:
            fe.nm ← fe.nm + 1
        endif
        fe.b ← (fe.b << 1) | 1          // add match
        (i,j) ← (i+1, j+1)
    endwhile
    fe.row ← i - 1                      // common prefix ends at u[..i-1]
    return fe
```

Optimizing the output sensitive alg

Two Strategies:

2) • trim the front-entries based on the observation that among the entries for the same front (same d-value) some may be far behind others in terms of the number of aligned symbols (sum of the rows & columns reached)

- front-head be a front-entry front$(j-i, d) = i$, s.t. $i+j$ is maximal $\Leftrightarrow$ sum of the length of the aligned prefixes is max
- L = maximum distance to the front-head
- $\forall$ front$(j'+i', d) = i'$, $i'+j' < i+j-L$ are trimed
- $i+j \;(=)\; 2i+k$, if front$(j-i, d) = i$, $k = j-i$

⇒ Both strategies are combined for trimming

```
trim2 (fe, didx, rq, fhd, L):
    if re.row < rq:
        return true          // trim poor regard diagonal quality
    endif
    if 2·fe.row + didx < 2·fhd.row + fhd.didx − L:
        return true          // too far behind
    eff
    return false
```

GLK

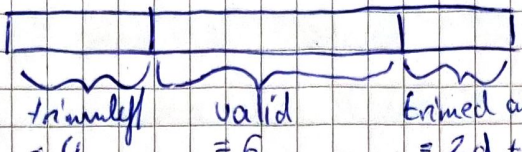
Optimize the output sensitive alg.

→ trimming from both ends.

→ Computation ends:

if the end of the sequence u and v is reached (edit-distance)

or if all entries of a band have been trimmed (edits for prefixes)



Alg: Optimized output sensitive alg. In: u, v, Minimum Percent Identity $minid \in [0,100]$, rq, L, depending on $minid$

Out: if $edist_q(u,v) \leq (|u|+|v|) \cdot \frac{100-minid}{100}$ then $edist_q(u,v)$, else $m+n-1$

```
fast-edit-trim(u, v, rq, L)
    (m, n, trimleft, valid) ← (|u|, |v|, 0, 1)
    for d ← 0 upto max{n, m} do
        fhd ← ⊥
        for h ← -d + trimleft upto -d + trimleft + valid - 1 do
            FE(h,d) ← evaluate-frontentry(u, v, h, d)
            if fhd = ⊥ or 2·fhd.row + fhd.didx < 2FE(h,d).row + h then
                fhd = FE(h,d)                    // update front head
                fhd.didx = h
            endif
        endfor
        trim ← 0
        for h ← -d + trimleft upto -d + trimleft + valid - 1 do
            if trim?(FE(h,d), h, rq, fhd, L) then trim++ else break
            endif
        endfor
        valid ← valid - trim
        trimleft ← trimleft + trim
        trim ← 0
        for h ← -d + trimleft + valid - 1 downto -d + trimleft do
            if trim?(FE(h,d), h, rq, fhd, L) then trim++ else break
            endif
        endfor
        valid ← valid - trim
        if valid = 0 then return m+n-1 endif   // no edit distance :(
        if -d + trimleft ≤ n - m ≤ -d + trimleft + valid - 1 then
            if FE(n-m, d).row = m then
                return d                       // edit distance :)
            endif
        endif
        valid ← valid + 2                      // new d => 2 new valid entries
    endfor
```

GIK

Optimizing the output structure alg
Using the output construction alg for clustering many old seq's

Metagenomics : • study of the DNA of unicellular organisms

• Metagenome : entire genetic info of community of organisms

Task : Determine operational taxonomic units (OTUs) by clustering sequences with identity $\geq 97\%$

Two seq's same cluster if $id(u,v) \geq \sigma$ for some minimum identity percentage value

$$id(u,v) = 100\left(1 - \frac{edist_\zeta(u,v)}{\ell}\right), \quad \ell = \frac{|u|+|v|}{2}$$

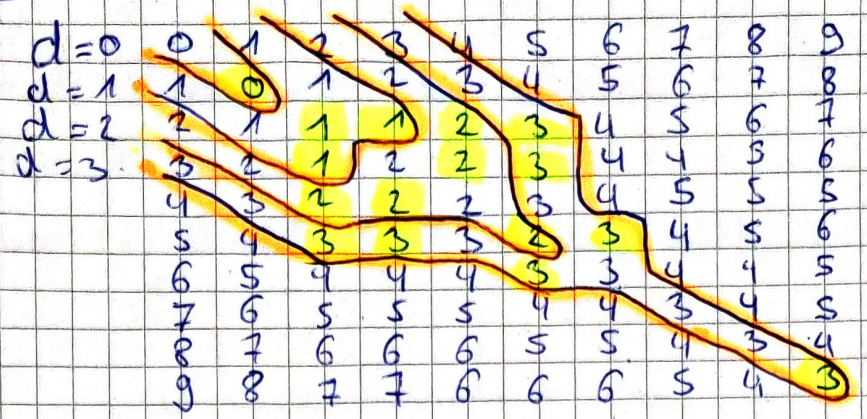
$$id(u,v) \geq \sigma \iff edist_\zeta(u,v) \leq d_{max} = \left\lfloor \left(1 - \frac{\sigma}{100}\right) \cdot \ell \right\rfloor$$

↳ so for each pair determine $edist_\zeta(u,v) \leq d_{max}$

→ modify alg s.t. alg. iterates until d_{max}

↳ if $trace(u-m, d) = m$, $d \leq d_{max}$ → return true, else false

(3.4)



$0 \leq d \leq 3$, front (k, d)

GK

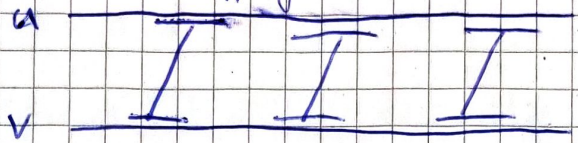
Chaining of Matches

Anchor based sequence comparison

Three Phases:

Phase 1: Computation of matches (substrings with small edists)

Phase 2: Computation of a highest-scoring chain of colinear non-overlapping matches: the anchors form the basis of the Alignment



// match chaining problem

Phase 3: Alignment of the region between the anchors

Note: also for ≥ 2 sequences possible

Match Chaining Problem

Combinatorial Chaining Problem of Phase 2: finding a highest-scoring chain of colinear non-overlapping matches

Colinear:

Two matches are colinear if the order of their respective matching substrings is the same in both genomes.
(non-crossing)

Two matches overlap if the sequences involved in the matches overlap in at least one of the genomes

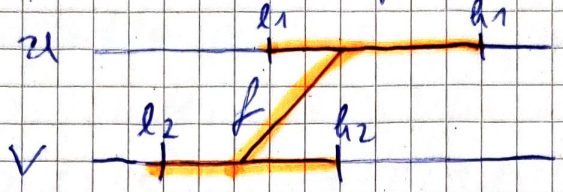
GIK

Match Chaining Problem

Match:

A match f is given by two pairs $s(f) = (l_1, l_2)$ and $e(f) = (h_1, h_2)$ s.t. the strings $u[l_1 .. h_1]$ and $v[l_2 .. h_2]$ have a small edit distance and a sufficient length.

→ $s(f)$: start point of f, $e(f)$: end point of f



→ then $u[l_1 .. h_1] \overset{!}{=} v[l_2 .. h_2]$

e.g.: maximal unique matches, maximal exact matches, exact q-grams

→ For exact matches one uses the length as weight

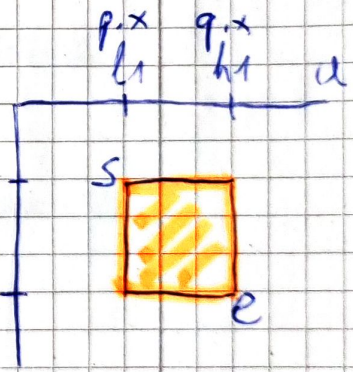
→ For inexact matches, the weight is often defined as the score of the optimal alignment of the matching strings

→ weight $(f) > 0$

→ Match is a rectangle in a $(n+1) \times (m+1)$ grid

$s(f) = (l_1, l_2)$, $e(f) = (h_1, h_2)$

↳ rectangle $[l_1 .. h_1] \times [l_2 .. h_2]$



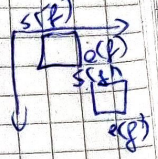
→ More general $R(p, q)$, p upper left corner, q lower right corner

GIK

Match Chaining Problem

$\mathcal{M}$ be the given set of matches. We define binary relation $\ll$ on $\mathcal{M}$ by $f \ll f'$, iff

$$e(f).x < s(f').x \quad \text{and} \quad e(f).y < s(f').y$$



Chain of colinear non-overlapping matches:

$\mathcal{M}$ be a set of weighted matches. A chain of colinear non-overlapping matches is a sequence of matches $f_1, \dots, f_\ell$ s.t.

- $\rightarrow f_i \in \mathcal{M}, \quad \forall i \in [1, \ell]$
- $\rightarrow f_i \ll f_{i+1}, \quad \forall i \in [1, \ell-1]$

The score of C is $\text{score}(C) = \sum_{i=1}^{\ell} \text{weight}(f_i)$

Matching Problem:

Given a set $\mathcal{M}$ of weighted matches, the match-chain problem is to determine a chain of maximal score. $\rightarrow$ optimal chain

$\rightarrow$ An optimal chain ends with a match f s.t. there is no match f' : $f \ll f'$

$\rightarrow$ An optimal chain begins with a match f' s.t. there is no match f : $f \ll f'$

$\Rightarrow$ Global Chaining Problem close to maximality on both ends

GfG

A simple chain problem

Construct a weighted DAG $G = (V, E)$ s.t.:

→ set V of vertices is $\mathcal{M}$

→ set E of edges:

There is an edge $f \to f'$ with weight(f'), iff $f \ll f'$

→ first match f of a chain gets weight weight(f)

⇒ Find path of maximum score in graph = optimal chain

→ Idea: to divide the set of all chains according to the last match they consist of.

↳ $\forall f'$ we have → set of chains ending with f'

→ score(f') = maximum score of all chains ending with f'

↳ consider all chains ending with some match f which may precede f' and add weight(f')

$$\Rightarrow \text{score}(f') = \max(\{0\} \cup \{\text{score}(f) \mid f \in \mathcal{M}, f \ll f'\}) + \text{weight}(f')$$

↳ for $f \in \mathcal{M}, f \ll f'$

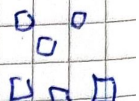
GdK

Simple Chain Algorithm

Alg° In°: $M = \{f_1, \dots, f_b\}$ ordered by x-coordinate of end-point

Out°: optimal chainscore, last element, precursor table

```
overallmaxscore ← 0
lastmatch ← ⊥                    // last match
for j ← 1 upto b do
    maxscore ← weight(f_j)       // chain with only f_j
    precursor[f_j] ← ⊥
    for i ← 1 upto j-1 do        // since order by x
        if f_i << f_j and maxscore < score(f_i) + weight(f_j) then
            maxscore ← score(f_i) + weight(f_j)      // extend by f_i
            precursor[f_j] ← f_i
        endif
    endfor
    score(f_j) ← maxscore
    if overallmaxscore < maxscore then
        overallmaxscore ← maxscore
        lastmatch ← f_j
    endif
endfor
return (overallmaxscore, lastmatch, precursor)
```



Runtime $O(b^2)$

Space $O(b)$

An efficient match chain problem

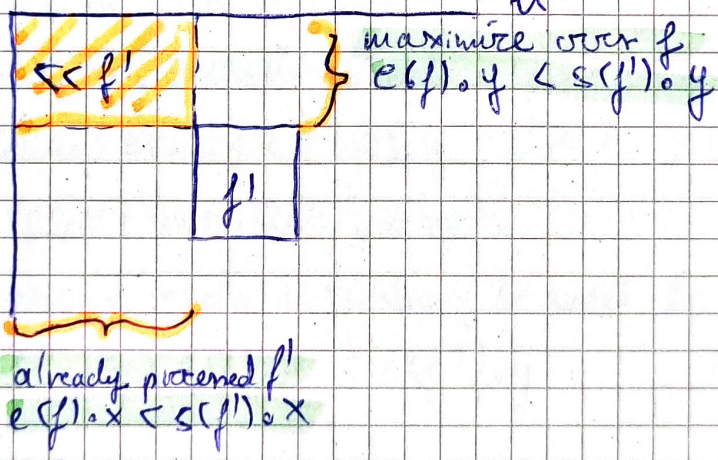
→ score(f'), need to maximize over all scores of f, f << f'

→ In geometric terms: max over all matches in the rectangle $R((0,0),(s(f').x - 1, s(f).y - 1))$

↳ this two dimensional search problem can be reduced to 1-D search problem by processing the matches in ascending order of their x-coordinate.

→ only consider matches where end point is strictly to the left of the start point of the current fragment w.r.t. to the x-coordinate

→ Idea: maintain a relevant subset of the previous matches in a dictionary structure, ordered by the y-coordinate



GUK

efficient match chaining alg.

→ Alg considers each match twice, once w.r.t. start point and once w.r.t. end point

→ $M_{S,E} = \{(f,0) \mid f \in M\} \cup \{(f,1) \mid f \in M\}$
(0: s, 1: e)

→ projection φ: $\varphi(f,0) = s(f).x$
$\varphi(f,1) = e(f).x$

→ $M_{S,E}$ is sorted $\prec_x$

$(f,a) \prec_x (f',a') \Leftrightarrow \varphi(f,a) < \varphi(f',a')$ or
$(\varphi(f,a) = \varphi(f',a'),\ a < a')$

↳ matches are sorted in ascending order w.r.t. their x-coordinate

→ $M_{S,E}$ is ordered according to $\prec_x$ then consider each match twice one time at start and end according to y-coordinate

→ Case 1°: $(f',0)$ is currently processed.
only consider $e(f).y < s(f').y$
→ if f: $score(f') = \max\{score(f) \mid \dots\} + weight(f')$
→ else: $score(f') = weight(f')$

→ Case 2°: $(f',1)$ is currently processed.
store processed matches in dictionary D sorted by $\prec_y$ (ascending)

$f \prec_y f'' \Leftrightarrow e(f).y < e(f'').y$

order $\prec_y$ in the processed matches implies their order w.r.t. to their score

↳ $e(f).y \leq e(f'').y$ and $score(f) \geq score(f'')$

↳ Then optimal chain does not include f''

→ Prevent such matches:

I) f' is only inserted in D, if no predecessor or if the predecessor has a smaller score than f'

II) Once f' is inserted in D, one follows the chain of successors of ($\prec_y$) f' and deletes all those which have a smaller or equal score than f'

$f \prec_y f''$ ⇒ $score(f) \leq score(f'')$ Because of this

GVK

Efficient Match chain Alg

Dictionary D:

- → insert(D, f): if not f in D insert f into D
- → delete(D, f): remove f from D
- → pred(D, p): determine largest match $f'' \in D(\prec_y)$ satisfying $e(f'').y < p$. or $\perp$ if no match
- → succ(D, p): determine smallest match $f'' \in D(\prec_y)$ satisfying $p < e(f'').y$. or $\perp$ if no match

// Dictionary search $O(\log b)$ time

// loop $O(bt \log bt)$

// preprocess $O(b \log b)$

↳ Runtime $O(b \log b)$
Space (b)

Alg:

Compute $M_{S,E}$ and sort it according to $\prec_x$
$D \leftarrow \emptyset$
for all $(f', a) \in M_{S,E}$ in sorted order do
  if $a = 0$ then // process start pair; D read only
    $f \leftarrow$ pred(D, $s(f').y$)
    if f is $\perp$ then
      score(f') $\leftarrow$ weight(f')
      precursor[f'] $\leftarrow \perp$
    else
      score(f') $\leftarrow$ weight(f') + score(f)
      precursor[f'] $\leftarrow f$
    endif
  else // process end pair, f' possibly added to D
    if pred(D, $e(f').y$) = $\perp$ or score(pred(D, $e(f').y$)) < score(f') then
      insert(D, f') // now f' follows its predecessor in D
      $f'' \leftarrow$ succ(D, $e(f').y$)
      while $f'' \neq \perp$ and score(f') ≥ score(f'') do
        tmp $\leftarrow f''$
        $f'' \leftarrow$ succ(D, $e(f'').y$)
        delete(D, tmp)
      end while
    endif
  endif
end

GdK -Tut

(9.1) Matches $f \& f'$, $f \ll f'$:

$$gap\text{-}1(f, f') = s(f').x - e(f).x - 1 + s(f').y - e(f).y - 1$$

- gap-1 = # pixels zwischen zwei kollinearen nicht überlappenden Treffern.

Sei $C = f_1, f_2, \ldots, f_k$ eine Kette von kollinearen nicht überlappenden Treffern. $f \in M$.

- Lokale Score:

$$localscore(C) = \sum_{i}^{k} weight(f_i) - \sum_{i=2}^{k} gap\text{-}1(f_{i-1}, f_i)$$

↳ verlangt Lücken zwischen Matches

- Lokale Chaining-Problem für M:
  find a chain C, s.t. $localscore(C) \geq localscore(C')$, $\forall C'$.
  ↳ C is called optimal local chain

- Alg:
  für den j-ten Treffer f_j (ordered by x-coordinate des Endpunktes) nur die vorherigen Treffer f_i mit $\max\{1, j-p\} \leq i \leq j-1$.
  ↳ maximal p vorherige Treffer für jeden Treffer f_j betrachtet.
  b Treffer $\Rightarrow$ Runtime $O(pb)$

GBK – Ü4

(4.1)

Alg.:

```
local-chaining-problem (matchcollection):
    overallmaxscore = 0
    bestmatch = None
    p = 50

    for j in range (0, matchcollection.size()):
        maxscore = matchcollection[j].weight()   // no-precursor
        matchcollection[j].prec_set (None)

        for i in range (max(0, j-p), j):
            if matchcollection[i].simple-precedes (matchcollection[j]):
                if is-new-max (matchcollection[i], matchcollection[j], maxscore):
                    maxscore = matchcollection[i].score() + matchcollection[j].weight()
                               - matchcollection[i].gap*1 (matchcollection[j])
                    matchcollection[i].prec_set (matchcollection[i])
        matchcollection[j].score_set (maxscore)   // update score
        if overallmaxscore < maxscore:
            overallmaxscore = maxscore
            bestmatch = matchcollection[j]
    precursor = get_all_precursor (bestmatch)   // get chain
    return precursor, overall max
```

$O(p \cdot b)$